Does the implementation compute what the spec computes?

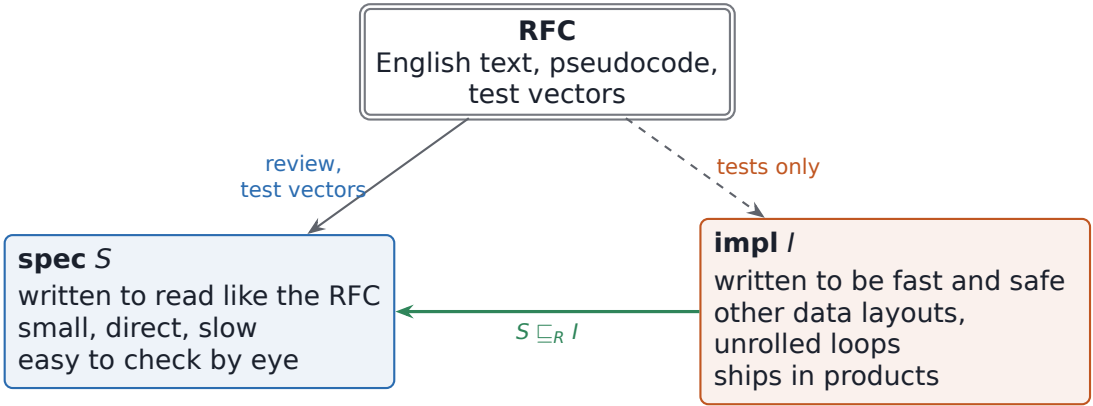
A method to prove refinement between two Rust programs in Lean, through Aeneas

ref-impl-simulation

November 2026

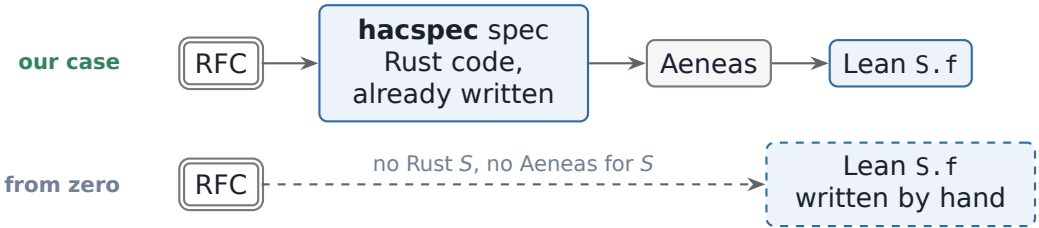


Two Rust programs for one RFC



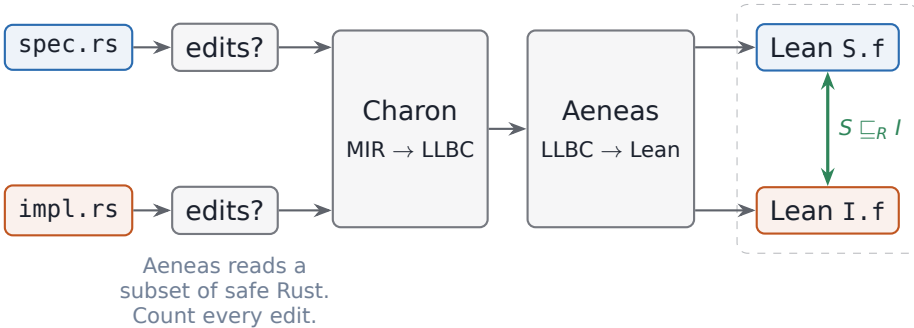
- People can read S and compare it with the RFC.
- It is more difficult to do the same for I.
- A proof that I computes what S computes moves the trust from S to I.
- We write $S \sqsubseteq_R I$: I refines S under a relation R.

We do not start from zero: hacspec



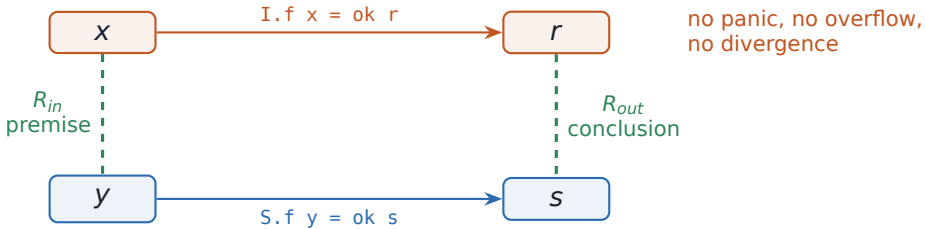
- hacspec, and the language of the hax toolchain, are subsets of Rust for executable specs.
- So *S* exists before we start. We reuse it and run Aeneas on it, as on *I*.
- Its fit to the RFC is trusted, and test vectors check it. A spec modelled on the real code (Ristretto follows dalek) cannot find an error that both share.
- Without such a spec, we can write *S* directly in Lean and skip Aeneas for *S*.
- The price: old hacspec does not go through Aeneas as is. It needs a port or a shim, and both are trusted.

The pipeline



- Aeneas turns each Rust function into a pure Lean function in a `Result` monad.
- The question about 2 Rust programs becomes a question about 2 Lean functions.

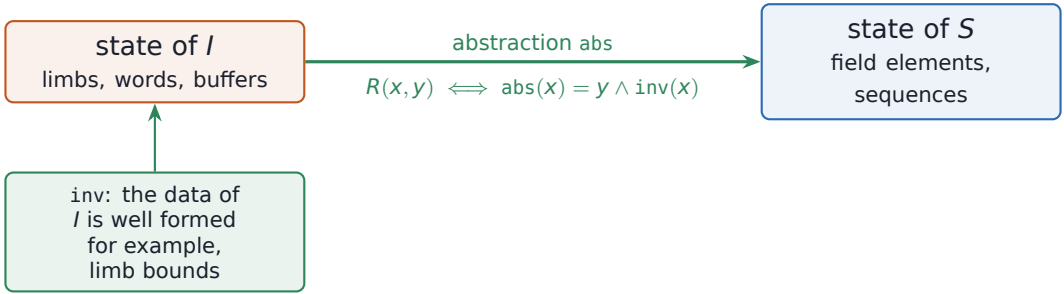
The theorem $S \sqsubseteq_R I$: a commuting square




```
theorem I_refines_S (x : I.In) (y : S.In) (h : R_in x y) :
    I.f x ⦃ r => ∃ s, S.f y = .ok s ∧ R_out r s ⦄
```

- $R = (R_{in}, R_{out})$.
- $⦃\ ⦄$ is the total-correctness triple of the Aeneas library.
- Both functions are deterministic, so the results agree whenever R_{in} holds.

The relation maps *I* data to *S* data



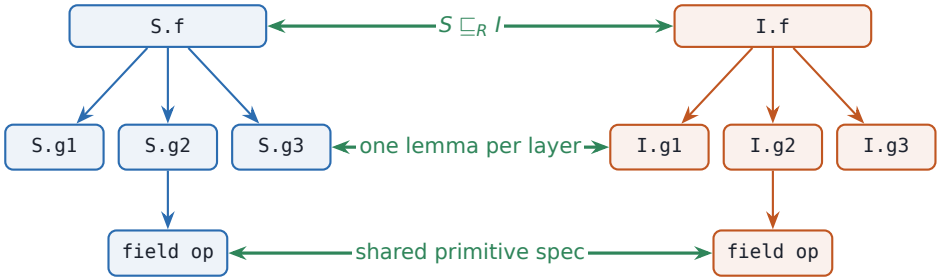
ChaCha20
equal lists of 32-bit words

X25519
5 limbs of 51 bits map to `ZMod (2^255 - 19)`, limbs stay below 2^{52}

HPKE
equal byte strings, a map between algorithm identifiers

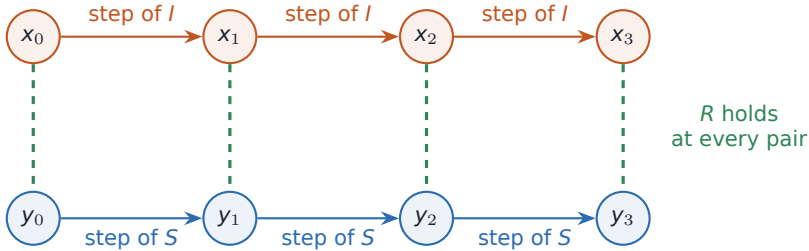
- Each study defines its own relation.
- The invariant lets the next step start.

The proof follows the call path



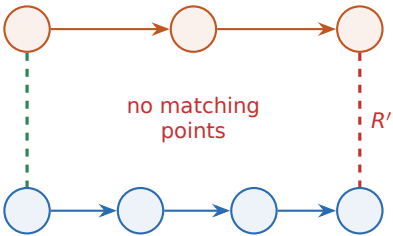
- Split each side at its calls.
- Prove one lemma for each pair of calls that match.
- The main theorem composes the lemmas.
- The leaves are often shared primitives: a field operation, a hash, a byte copy.

Lockstep where the control flows match



- One rule closes each square: take one step on each side, then relate the results.
- The proof is mechanical, so a tactic can produce it.
- A step is a call, a field operation or a shared primitive.

Where the control flows differ



The proof needs a hand-written argument

a different formula: a ring identity

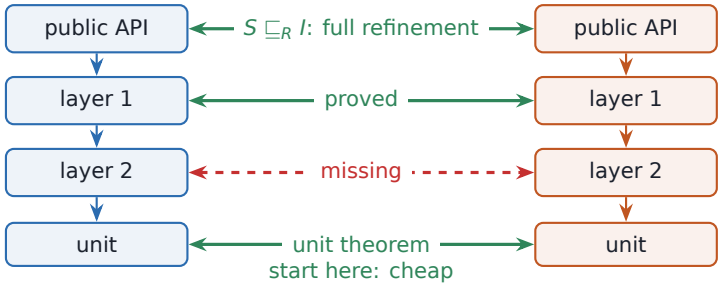
loops of different length: an extra step

states equal only up to a factor: a loop invariant with a weaker relation R'

the end of the loop restores R

- This part does not come from a tactic.
- It is where most of the effort goes.

From one unit to full refinement



- A unit tests the method at a low cost.
- Full refinement relates the public API of I to the public API of S.
- It needs one more result for each layer between the unit and the API.

What a theorem trusts

the theorem, checked by the Lean kernel	
premises: in the statement	read the statement
trusted definitions: Lean models, Rust ports and shims	review
axioms: `#print axioms`, beyond the standard 3	list and explain
S itself: does it match the RFC?	review, test vectors
tools: rustc MIR, Charon, Aeneas and its library, Lean	tools

- `#print axioms` does not show trusted definitions. Each study lists them by hand.
- Rules: no `sorry`, no `maxHeartbeats` bump in a proof.

Three instances of the method

	spec *S*	**impl** *I*	**proved**
RFC 8439 ChaCha20	hax-style spec	libcrux HACL	block core (unit)
RFC 7748 X25519	**hacspec, ported to plain Rust**	**x25519-dalek**	**full refinement**
RFC 9180 HPKE	hacspec, behind a shim	hpke-rs	key schedule (unit)

Next: X25519 in detail

- *S* is `x25519_scalarmult`, *I* is `x25519_dalek::x25519`.
- The theorem has no premises.
- Lockstep closes the ladder step.
- The ladder loop needs a hand-written invariant.